AUTOMATED TIMETABLE GENERATION

SOFTWARE REQUIREMENTS SPECIFICATION

DEPARTMENT OF COMPUTER SCIENCE
Forman Christian College (A Chartered University)

TITLE PAGE

Automated Academic Timetable Generation

Final Year Project Team:

Muhammad Zeeshan
Alisha Saleem
Abial Zafar
Sheril Bobby

Project Advisor: Dr. Nazim Ashraf
Project Secondary Advisor: Sharoon Nasim
Submission Date: 05th January, 2025

Signatures:

Project Adviser: _____
Muhammad Zeeshan: _____
Sheril Bobby:_____

Project Secondary Adviser:_____
Alisha Saleem:_____
AbialZafar:_____

Table of Contents

Table of Contents iii
Revision History iv
1. Introduction and Background 1
1.1 Product (Problem Statement) 1
1.2 Background 1
1.3 Scope 1
1.4 Objective(s)/Aim(s)/Target(s) 1
1.5 Challenges 1
1.6 Learning Outcomes 1
1.7 Nature of End Product 1
1.8 Completeness Criteria 1
1.9 Business Goals 1
1.10 Related Work/ Literature Survey/ Literature Review 2
1.11 Document Conventions 2
2. Overall Description 3
2.1 Product Features 3
2.2 User Classes and Characteristics 3
2.3 Operating Environment 3
2.4 Design and Implementation Constraints 3
2.5 Assumptions and Dependencies 3
3. Functional Requirements 4
3.1 Name of Use-Case 1 4
3.2 Name of Use-Case 2 (and so on) 4
3.3 Requirements Analysis and Modeling 4
4. Nonfunctional Requirements 5
4.1 Performance Requirements 5
4.2 Safety Requirements 5
4.3 Security Requirements 5
4.4 Additional Software Quality Attributes 5
5. Other Requirements 5
6. Revised Project Plan 5
7. References 6
Appendix A: Glossary 7
Appendix B: IV & V Report 8
(Independent verification & validation) 8

Revision History

Name	Date	Reason For Changes	Version

Introduction and Background

Product (Problem Statement)

Academic institutions face a complex logistical challenge in generating class timetables that meet numerous hard and soft constraints. At Forman Christian College, the Intermediate Program consists of five academic streams, Medical, Engineering, Computer Science, Humanities, and Commerce, each with approximately 15 sections, totaling 150 class groups across Year 1 and Year 2. Each section follows a specific subject structure, and timetables must align with:

- *Subject-wise lecture allocations*
- *Teacher availability and workload limits*
- *Classroom capacity and availability*
- *Shift timings (morning and afternoon)*
- *Double-period lab requirements*
- *Special substitutions such as Ethics replacing TQ for non-Muslim students*
- *Merge/split constraints for specific subjects*
- *Fortnightly sessions (e.g., Music)*

Currently, timetable creation is managed manually by academic coordinators using a trial-and-error approach, which is time-consuming, unscalable, and error-prone. This leads to frequent:

- *Teacher or room schedule conflicts*
- *Unbalanced teaching workloads*
- *Poor classroom utilization*
- *Delays in releasing the final schedule*

With over 4,500+ lectures to be scheduled per week, 50+ faculty members, and 30+ rooms/labs, the manual process results in operational inefficiencies that negatively impact both staff and students.

Who Needs It?

The primary stakeholders and their needs are as follows:

Stakeholder	***Need/Benefit***
Academic Coordinators	*Efficient and constraint-compliant timetable generation*
Teachers	*Balanced teaching workload; no clashes or excessive consecutive lectures*
Students	*Conflict-free class schedules and optimized learning flow*
Administration Department	*Better utilization of rooms and faculty; reduction in scheduling workload*

○ Background

*The domain is **Automated Academic Timetable Generation**. This involves solving a **complex combinatorial maximization dilemma** encountered in educational institutions, specifically the **Intermediate Program at Forman Christian College (FCC)**. The process requires assigning **Subjects**, **Teachers**, **Rooms**, and **Class Sections** to specific **Lecture Slots** (time/day) while satisfying numerous conflicting and non-conflicting constraints.*

*The current system at FCC is handled **manually** using a trial-and-error method. This traditional approach is **time-consuming, error-prone**, and leads to problems like scheduling conflicts, unbalanced teacher assignments, and inefficient resource use, especially given the need to schedule over **4,500+ lectures weekly** for approximately **150 distinct groups**. The system must account for two academic years (1st and 2nd) , five academic streams (Medical, Engineering, ICS, Arts, ICOM) , and two shifts (Morning and Afternoon).*

○ Scope

*The scope of the Slotify product is a **web-based timetable generation system** designed to **automate and optimize** the academic scheduling for the **Intermediate Program at FCC**.*

The product's scope covers:

- ***Data Management:** Full management and configuration of all core academic entities, including **Teachers, Rooms, Subjects, Departments, Groups, Sections, Sessions (Years), and Shifts**.*
- ***Core Functionality:** Implementation of the **Genetic Algorithm (GA)** using **OptaPlanner** to intelligently generate a **conflict-free timetable**.*
- ***Constraint Handling:** Satisfying complex institutional rules, including **hard constraints** (e.g., room capacity, teacher workload limits, no double-booking) and **soft constraints** (e.g., balanced workload, merged classes, double-period labs, specific timings).*
- ***User Interaction:** Providing user interfaces for **secure login**, **visualizing schedules** (teacher-wise, class-wise, room-wise), and allowing for **manual overrides** by authorized Academic Coordinators.*
- ***Reporting:** Providing an **Analytics Dashboard** and **Export functionality** (PDF, Excel) for the generated timetables.*

*The project **does not** include:*

- *Scheduling for programs outside the FCC Intermediate Program.*
- *Integration with external Human Resources or Student Information Systems beyond the defined data model.*

○ Objective(s)/Aim(s)/Target(s)

*The primary goal of Slotify is to **facilitate and ease the burden of the current manual timetable arrangement setup**.*

The main objectives to be achieved are:

- ***Automation and Optimization:*** *To successfully design and implement a scheduling engine using **Genetic Algorithms (OptaPlanner)** that can automatically generate **conflict-free** and **optimized** academic timetables.*
- ***Time Reduction:*** *To alleviate the **long time taken** for the scheduling task, replacing the current trial-and-error method with an automated solution.*
- ***Constraint Satisfaction:*** *To guarantee that the generated timetable adheres to all **hard constraints** (e.g., teacher availability, room capacity, workload limits) and minimizes violations of **soft constraints** (e.g., workload balance, reducing scheduling gaps).*
- ***Resource Utilization:*** *To improve the **incapacitated use of resources** (rooms, teachers) through intelligent assignment and provide analytics on room utilization and teacher load.*
- ***Scalability and Marketability:*** *To create a robust, scalable, and customizable solution that can be **marketable** and **extended** to other educational institutions in Pakistan and globally.*
- ***User Interface Provision:*** *To provide a **web-based system** (Spring Boot + React.js) with features for **manual adjustments, analytics, and export functions**.*

○ Challenges

Automated timetable generator Construction of an automated timetable generator of FCC is associated with a number of challenges which are real and practical. The greatest obstacle is the amount of constraints we must observe simultaneously, teacher workloads, room capacities, merged classes, lab requirements and shift timings. A single adjustment made in one place can cause a domino effect over the whole schedule, thus the system must be intelligent enough to modify itself without disruption to the rest of the schedule.
The other difficulty is that the reasoning of human beings cannot be translated into hard rules that can be comprehended by an algorithm. The decision-making process of coordinators is based on instinct or experience, yet our system needs to interpret it as accurately and as consistently as possible. It is also hard to deal with incomplete or last minute data changes, since the generator should be flexible and at the same time maintain correctness.
Lastly, the massive amount of data, 3000 or more lectures weekly, 100 or more teachers, and dozens of rooms, must be optimized in a fast and dependable way. One of the more challenging technical challenges of the project is to ensure the system will be able to work on this scale without including lengthy waiting times or errors in the system.

○ Learning Outcomes

It will be a project that will assist us not only in developing technically but also in the way we think about the real-world systems. We will have a practical experience in constraint modeling, optimization methodologies and the complexities of working with such tools as OptaPlanner. These are the skills that extend beyond this project and can be useful in the industries where scheduling, allocation of resources, or automated planning is concerned.

On the software engineering front, we will get to know how to architect an entire system, starting with requirements, all the way to deployment, with a current stack, which incorporates Spring Boot, PostgreSQL, and React. We will also feel more at ease with the team arrangement, sharing of duties, module integration, and checking each other.

On the technical level other than that, we are going to know more about the academic structure of FCC and the decision making process by the administration. The experience enhances our communication, documentation and problem solving skills equipping us to work in actual work conditions

○ Nature of End Product

The end product will be a fully operational and web based system which will create entire academic timetables automatically. It will have a neat and user-friendly interface with which coordinators may enter the availability of teachers, rooms, classes sections, and subjects. By one step the system will generate a clash free schedule within all the constraints of the institution.
The users will be in a position to see the timetables in terms of the classes, the teacher or the room and make manual corrections whenever the need arises. The system will also contribute to the schedule being exported to PDF or excel as an easy means of distribution. As it were, the final product will be a user-friendly, all-inclusive instrument that will directly help FCC in handling bulk scheduling with accuracy and efficiency.

○ Completeness Criteria

The project will be declared complete when the system will produce valid timetable of each and every sections, teachers and rooms without conflict. All the core modules such as teacher management, room management, subject handling, and class setup should be all functional and linked to the backend.
The solution should also pass real test cases given by the Intermediate coordinators i.e. it should be able to handle actual FCC data such as merged classes, lab requirements and shift constraints. There should also be a user-friendly front end, good error management and the ability to export.
Lastly, the system must be installed ***on cloud platform (to be confirmed by sir)****, be accessible to authorized users and should be well documented and with user guide. As soon as the system provides the right timetables constantly and all the significant functions perform as planned, the project will have its completeness criteria.*

○ Business Goals

Efficient Use of Institutes Resources:
Our system will allow an institute the ability to align their resources according to their requirement. In this way, they will be able to efficiently make use of their available resoirces and our system will aid them in scheduling and assigning timeslots based on the resources they have.

Global Usability:
Forman Christian College's Intermediate is based on BISE Lahore and in turn so is our system which can adapt to all institutes of Intermediate Studies based in Lahore to be able to use them. The project seeks to develop a commercially viable timetable generation solution that can be adopted by other educational institutions, particularly those facing similar scheduling complexities.

Improvement in Academic Administration:
The system seeks to reduce the stress of manual period scheduling away from the academic coordinators enabling them to focus on other pressing and advancing matters.

Scalable and Extensible System:
The system we are developing is leaving room for updates that could add and expand upon what already exists in the system allowing the opportunity for a consistent system with expected updates, advancements and improvements over time.

Long Term Cost Reduction:
Automated Timetable Generator is able to reduce the cost of manually scheduling and repeated revisions impact on the schedule would be handled by the system.

○ Related Work/ Literature Survey/ Literature Review

Timetable generation in educational institutions is a complex combinatorial optimization problem that involves assigning courses, teachers, rooms, and timeslots while satisfying a wide array of constraints. Over the years, numerous approaches have been proposed to automate this scheduling task, leveraging optimization techniques such as Genetic Algorithms and constraint solvers like ***OptaPlanner****.*

Genetic Algorithms for Automated Timetable Generation

Genetic Algorithms have been widely recognized for their efficiency in solving complex scheduling problems. Thakare et al. present a system for automated timetable generation using GA, demonstrating significant improvements in both solution quality and computational efficiency. The study details the process of encoding timetable data into chromosome structures, generating an initial population that respects hard constraints, and applying genetic operators such as crossover and mutation to evolve better schedules over successive generations. The fitness function evaluates solution quality based on minimizing conflicts and satisfying soft constraints. Their results indicate that modified genetic operators can prevent new conflicts and accelerate convergence towards optimal timetables. The system also emphasizes user-friendly features like secure authentication and JavaScript-based validation for data entry, facilitating practical deployment.

Similarly, another study proposes a GA-based timetable generation system aimed at minimizing scheduling conflicts and producing near-optimal solutions in a timely manner. It highlights the limitations of classical linear or integer programming techniques for complex university schedules and advocates for the adaptive nature of GA in handling diverse constraints. The research elaborates on system architecture, including data encoding, population generation, and fitness evaluation, coupled with genetic operations that ensure continuous improvement of solutions. The system reportedly achieves solution qualities ranging from 60% to 80% of the optimal and offers advantages such as flexibility, reduced computational resources, and simplified scheduling processes.

OptaPlanner School Timetabling Use Case

OptaPlanner's school timetabling use case exemplifies the application of AI-based constraint solvers to optimize lesson scheduling by minimizing conflicts and improving teacher and student satisfaction. The system addresses hard constraints such as preventing simultaneous bookings of rooms, teachers, and students, while also optimizing soft constraints like minimizing gaps in teachers' schedules or encouraging lesson sequences in the same room. The use of advanced metaheuristic algorithms such as Tabu Search, Simulated Annealing, and Late Acceptance enables the solver to find near-optimal solutions efficiently. This open-source framework supports JVM languages and facilitates rapid development of custom scheduling solutions.

OptaPlanner and Spring Boot Integration

The Red Hat Build of OptaPlanner (version 8.29) provides a robust framework for solving constraint satisfaction problems, including school timetabling, by integrating metaheuristic algorithms with modern software technologies. The official documentation outlines a comprehensive step-by-step guide to developing a school timetable application using OptaPlanner with Spring Boot. This approach models domain entities such as Timeslot, Lesson, and Room, and enforce constraints via OptaPlanner's constraint streams to ensure

conflict-free timetables. The system includes practical features like solver configuration for balancing runtime and solution quality, logging for debugging, and database/UI integration for persistent storage and user interaction. Furthermore, the integration of monitoring tools such as Micrometer and Prometheus allows real-time tracking of application performance and health, facilitating scalable deployment in production environments.

Collectively, these studies highlight the effectiveness of metaheuristic algorithms and constraint solvers in automating timetable generation. Among the available tools, OptaPlanner stands out as a robust and production-ready solution, particularly due to its seamless integration with Spring Boot, which offers comprehensive tooling and monitoring capabilities. While Genetic Algorithms demonstrate flexibility and adaptability for complex scheduling tasks, the structured constraint-solving approach of OptaPlanner aligns more closely with our objective of building a scalable, rule-aware, and user-friendly timetable generation system. Based on these insights, we have chosen to focus on OptaPlanner as the core engine for this project.

○ Document Conventions

This document follows the standard IEEE formatting for Software Requirements Specifications. The following typographical conventions are used to ensure clarity and professional consistency:

- ***Boldface:*** *Used to highlight key terms, software entities (e.g.,* ***OptaPlanner****), user roles (e.g.,* ***Academic Coordinator****), and primary GUI elements like buttons or menu items (e.g.,* ***Generate Timetable****).*
- ***Monospaced Font:*** *Used to represent technical elements such as code snippets, file names, database table names (e.g., TimetableEntry), or specific API endpoints.*
- ***Numbered Lists:*** *Used to define sequential steps in a process, such as the Typical Course of Action within a Use Case.*
- ***Bullet Points:*** *Used for non-sequential lists of items, features, or constraints.*
- ***Table Captions:*** *All tables are numbered and labeled (e.g.,* ***Table 1: UC-1****) to provide clear context and easy cross-referencing.*

● Overall Description

○ Product Features

Slotify, the Automated Academic Timetable Generation System, is designed to automate and optimize the scheduling process for Forman Christian College's Intermediate Program. It offers a comprehensive suite of features to manage all aspects of timetable creation, conflict resolution, and data analysis.

- ***Intelligent Timetable Generation:*** *implemented via OptaPlanner, to generate conflict-free academic schedules that adhere to both hard (e.g., teacher availability, room capacity, double-period lab) and soft (e.g., balanced teacher workload, merged subjects) constraints.*
- ***Comprehensive Management Modules:*** *Interfaces for setting up and managing key academic entities:*
- ***Teacher Management:*** *Defines teacher profiles, maximum weekly load, and availability.*
- ***Room Management:*** *Allocates rooms/labs based on capacity and type.*
- ***Subject Management:*** *Handles subject-specific attributes like lab flags, weekly lecture counts, and merge eligibility.*
- ***Class Section/Group Setup:*** *Manages student groups, streams (Med, Eng, ICS, Arts, Icom), year (1st, 2nd), and shift timings (Morning/Afternoon).*

- ***Timetable Visualization and Override:*** *Provides an interactive interface to view, manually adjust, and accommodate exceptions in the generated schedules.*
- ***Multiple Views:*** *Offers teacher-wise, class-wise, and room-wise schedule displays.*
- ***Manual Edit:*** *Allows authorized users to override the generated schedule for specific lecture assignments.*
- ***Analytics and Reporting:*** *Includes an Analytics Dashboard for visual insights into key metrics, such as teacher load distribution, room utilization, and conflict detection.*
- ***Export Functionality:*** *Supports exporting the final timetables in various user-friendly formats, including PDF and Excel.*

○ User Classes and Characteristics

The system is designed for different classes of users within the educational institution, each with distinct needs and privileges.

User Class	*Characteristics*	*Need/Benefit*	*Priority*
Academic Coordinators (Users)	*Possess the highest security and privilege level; responsible for all setup, generation, and manual overrides; requires detailed understanding of all scheduling constraints.*	*Efficient, constraint-compliant timetable generation with full control and manual override capability.*	***Favored/High***
Teachers	*Use the system to view their personal, conflict-free schedules and their current weekly workload; requires secure login.*	*Balanced teaching workload and an accurate, conflict-free individual schedule.workload; requires secure login.*	***High***
Administration Department	*Requires access to analytics on resource usage (rooms, faculty) and final timetables for administrative planning; may have read-only access to most features.*	*Better utilization of rooms and faculty; reduction in scheduling workload and operational inefficiencies.*	***Medium***
Students	*Primary use is to view their class-section timetables, typically via a read-only public interface or mobile view (PWA support).*	*Conflict-free class schedules and optimized learning flow.*	***Medium***

○ Operating Environment

Slotify will operate as a modern, web-based application, requiring standard internet connectivity and a modern browser for full functionality.

Hardware Platform: *Cloud platforms such as* ***Render, Railway, AWS, or GCP*** *will be used for scalable and reliable hosting. The system is designed to run on containerized environments using* ***Docker****.*

Operating System: *The backend will operate on a standard Linux-based environment compatible with the chosen cloud providers. The frontend is platform-independent, requiring only a standard web browser on Windows, macOS, Android, or iOS devices.*

Software Components/Dependencies:

- ***Backend: Java*** *with* ***Spring Boot Framework****.*
- ***Database: PostgreSQL*** *for relational data persistence, with* ***Hibernate (JPA)*** *for ORM.*
- ***Scheduling Engine: OptaPlanner*** *(Genetic Algorithm/Constraint Solver).*
- ***Frontend: React.js, Next.js, and CSS****.*
- ***Optional Caching: Redis*** *for quick lookups.*
- ***Connectivity:*** *Requires HTTPS/REST API communication between the React/Next.js client and the Spring Boot server.*

○ Design and Implementation Constraints

The development of Slotify is bound by specific technical and institutional constraints.

- ***Technology Stack Mandate:*** *Must be implemented using the defined stack:* ***Spring Boot (Java)*** *for the backend and* ***React.js/Next.js/Tailwind CSS*** *for the frontend.*
- ***Optimization Engine:*** *The core scheduling logic must be implemented using* ***Genetic Algorithms (GA)*** *via the* ***OptaPlanner*** *framework.*
- ***Database Requirement:*** *The system must use* ***PostgreSQL*** *as the primary relational database.*
- ***Institutional Constraints (Hard and Soft):*** *The solution must satisfy all academic rules of Forman Christian College, including:*
    - ***Hard Constraints:*** *Preventing teacher/room double-booking, respecting maximum teacher workload, honoring room capacity, and ensuring double-period lab sessions are scheduled once per week.*
    - ***Soft Constraints:*** *Balancing teacher workloads and minimizing gaps in teacher schedules.*
- ***User Interface Standards:*** *The frontend must be* ***responsive*** *and support* ***Progressive Web App (PWA)*** *features for mobile access.*
- ***Security:*** *Must implement* ***secure authentication and authorization*** *for different user classes (Admin, Teacher).*

○ Assumptions and Dependencies

The success of the project is based on several key assumptions and external dependencies.

- ***Availability of OptaPlanner:*** *The project assumes that the* ***OptaPlanner*** *framework provides the necessary APIs and performance characteristics to efficiently solve the complex timetable problem within an acceptable timeframe.*
- ***Data Accuracy:*** *It is assumed that the initial data entered by the Academic Coordinators (e.g., teacher loads, room capacities, subject constraints) will be* ***accurate and complete****.*
- ***Institutional Cooperation:*** *The project depends on the* ***continued support and feedback*** *from the FCC Administration and Academic Coordinators for requirement refinement and system testing.*
- ***Cloud Service Reliability:*** *The deployment strategy depends on the* ***reliability and uptime*** *of the chosen cloud service providers (Render, Railway, AWS, or GCP).*
- ***Technology Stability:*** *The project assumes the major versions of the chosen open-source technologies (****Spring Boot, React, OptaPlanner****) will remain stable throughout the development lifecycle.*

• Functional Requirements

○ *Name of Use-Case 1: Login*

Identifier	*UC-1*
Purpose	*Allow users to authenticate and log into the system.*
Priority	*High*
Pre-conditions	*User has a registered account. System is operational.*
Post-conditions	*User is logged in and redirected to the Home Page.*

○ *Typical Course of Action*

S#	*Actor Action*	*System Response*
1	*User enters username and password.*	*System validates input format.*
2	*User clicks "Login" button.*	*System verifies credentials and grants access.*
3	*–*	*System redirects user to Home Page.*

○ *Alternate Course of Action*

S#	*Actor Action*	*System Response*
1	*User enters incorrect credentials.*	*System displays error: "Invalid username or password."*
2	*User clicks "Forgot Password".*	*System redirects to Forgot Password page (UC-3).*

○ *Name of Use-Case 2: Sign Up*

Identifier	*UC-2*
Purpose	*Register a new user account.*
Priority	*Medium*
Pre-conditions	*User is not registered.*
Post-conditions	*New account is created and user can log in.*

○ *Typical Course of Action*

S#	*Actor Action*	*System Response*
1	*User clicks "Sign Up" link.*	*System displays registration form.*

2	*User fills in details and submits.*	*System validates data and creates account.*
3	*–*	*System sends confirmation and redirects to login.*

- ***Alternate Course of Action***

S#	*Actor Action*	*System Response*
1	*User enters duplicate username/email.*	*System shows error: "User already exists."*
2	*User leaves required fields empty.*	*System prompts: "Please fill all required fields."*

- ***Name of Use-Case 3: Forgot Password***

Identifier	*UC-3*
Purpose	*Allow user to reset forgotten password.*
Priority	*High*
Pre-conditions	*User has a registered email.*
Post-conditions	*Password reset link is sent; user can set new password.*

- ***Typical Course of Action***

S#	*Actor Action*	*System Response*
1	*User enters registered email.*	*System validates email and sends reset link.*
2	*User clicks link in email.*	*System redirects to password reset page.*
3	*User sets new password and confirms.*	*System updates password and confirms success.*

- ***Alternate Course of Action***

S#	*Actor Action*	*System Response*
1	*User enters unregistered email.*	*System shows: "Email not found."*
2	*Reset link expires.*	*System prompts: "Link expired. Request again."*

- ***Name of Use-Case 4: Add Timetable***

Identifier	*UC-4*
Purpose	*Generate a new timetable using the scheduling engine.*
Priority	*High*

Pre-conditions	*User is logged in as Academic Coordinator.*
Post-conditions	*A new timetable is generated and saved.*

- ***Typical Course of Action***

S#	*Actor Action*	*System Response*
1	*Coordinator clicks "Generate Timetable".*	*System runs OptaPlanner engine.*
2	*–*	*System displays progress and final schedule.*
3	*Coordinator reviews and saves.*	*System stores timetable and updates Progress tab.*

- ***Alternate Course of Action***

S#	*Actor Action*	*System Response*
1	*Insufficient data (e.g., missing rooms).*	*System warns: "Incomplete data. Cannot generate."*
2	*Generation exceeds 30 minutes.*	*System cancels and suggests manual adjustments.*

- ***Name of Use-Case 5: Add/Edit Session***

Identifier	*UC-5*
Purpose	*Create or modify academic sessions (e.g., Year 1, Year 2).*
Priority	*Medium*
Pre-conditions	*User is Academic Coordinator.*
Post-conditions	*Session data is updated in the system.*

- ***Typical Course of Action***

S#	*Actor Action*	*System Response*
1	*User navigates to "Sessions" page.*	*System displays list of existing sessions.*
2	*User clicks "Add New" or "Edit".*	*System opens form for input.*
3	*User submits form.*	*System validates and saves changes.*

- ***Alternate Course of Action***

S#	*Actor Action*	*System Response*
1	*User tries to delete a session in use.*	*System prevents deletion and shows warning.*

2	*User enters duplicate session name.*	*System shows: "Session already exists."*

- ***Name of Use-Case 6: Add/Edit Shifts***

Identifier	*UC-6*
Purpose	*Manage shift timings (Morning/Afternoon).*
Priority	*Medium*
Pre-conditions	*User is Academic Coordinator.*
Post-conditions	*Shift data is updated and reflected in scheduling.*

- ***Typical Course of Action***

S#	*Actor Action*	*System Response*
1	*User goes to "Shifts" page.*	*System shows existing shifts.*
2	*User adds or edits shift details.*	*System validates and saves.*
3	*–*	*System updates related timetable constraints.*

- ***Alternate Course of Action***

S#	*Actor Action*	*System Response*
1	*User sets overlapping shift times.*	*System warns: "Shifts cannot overlap."*
2	*User deletes a shift used in timetable.*	*System blocks deletion and suggests reassignment.*

- ***Name of Use-Case 7: Add/Edit Groups***

Identifier	*UC-7*
Purpose	*Manage student groups (streams: Med, Eng, ICS, Arts, ICOM).*
Priority	*High*
Pre-conditions	*User is Academic Coordinator.*
Post-conditions	*Group data is updated and available for scheduling.*

- ***Typical Course of Action***

S#	*Actor Action*	*System Response*
1	*User navigates to "Groups" page.*	*System displays existing groups.*

2	*User adds/edits group information.*	*System validates and saves.*
3	*–*	*System updates Progress tab completion.*

- ***Alternate Course of Action***

S#	*Actor Action*	*System Response*
1	*User tries to assign non-existent stream.*	*System shows: "Invalid stream selected."*
2	*User leaves required fields blank.*	*System prompts: "Please fill required fields."*

- ***Name of Use-Case 8: Add/Edit Subjects***

Identifier	*UC-8*
Purpose	*Manage subject details (lab flags, lecture counts, merge eligibility).*
Priority	*High*
Pre-conditions	*User is Academic Coordinator.*
Post-conditions	*Subject data is updated for timetable generation.*

- ***Typical Course of Action***

S#	*Actor Action*	*System Response*
1	*User goes to "Subjects" page.*	*System lists subjects.*
2	*User adds/edits subject attributes.*	*System saves changes and updates constraints.*
3	*–*	*System reflects changes in scheduling options.*

- ***Alternate Course of Action***

S#	*Actor Action*	*System Response*
1	*User sets invalid weekly lecture count.*	*System warns: "Invalid lecture count."*
2	*User deletes subject used in timetable.*	*System prevents deletion and suggests archive.*

- ***Name of Use-Case 9: Add/Edit Sections***

Identifier	*UC-9*
Purpose	*Manage class sections within groups.*
Priority	*High*

Pre-conditions	*User is Academic Coordinator.*
Post-conditions	*Section data is updated and used in scheduling.*

- ***Typical Course of Action***

S#	*Actor Action*	*System Response*
1	*User navigates to "Sections" page.*	*System displays sections per group.*
2	*User adds/edits section details.*	*System validates and saves.*
3	*–*	*System updates Progress tab.*

- ***Alternate Course of Action***

S#	*Actor Action*	*System Response*
1	*User assigns section to non-existent group.*	*System shows error: "Group not found."*
2	*User enters duplicate section code.*	*System warns: "Section code must be unique."*

- ***Name of Use-Case 10: Add/Edit Departments***

Identifier	*UC-10*
Purpose	*Manage academic departments.*
Priority	*Low*
Pre-conditions	*User is Academic Coordinator.*
Post-conditions	*Department data is updated.*

- ***Typical Course of Action***

S#	*Actor Action*	*System Response*
1	*User goes to "Departments" page.*	*System lists departments.*
2	*User adds/edits department info.*	*System saves changes.*
3	*–*	*System updates related teacher assignments.*

- ***Alternate Course of Action***

S#	*Actor Action*	*System Response*
1	*User tries to delete department with teachers.*	*System prevents deletion.*

2	*User leaves name blank.*	*System prompts: "Department name required."*

- ***Name of Use-Case 11: Add/Edit Rooms***

Identifier	*UC-11*
Purpose	*Manage room/lab details (capacity, type, availability).*
Priority	*High*
Pre-conditions	*User is Academic Coordinator.*
Post-conditions	*Room data is updated for scheduling.*

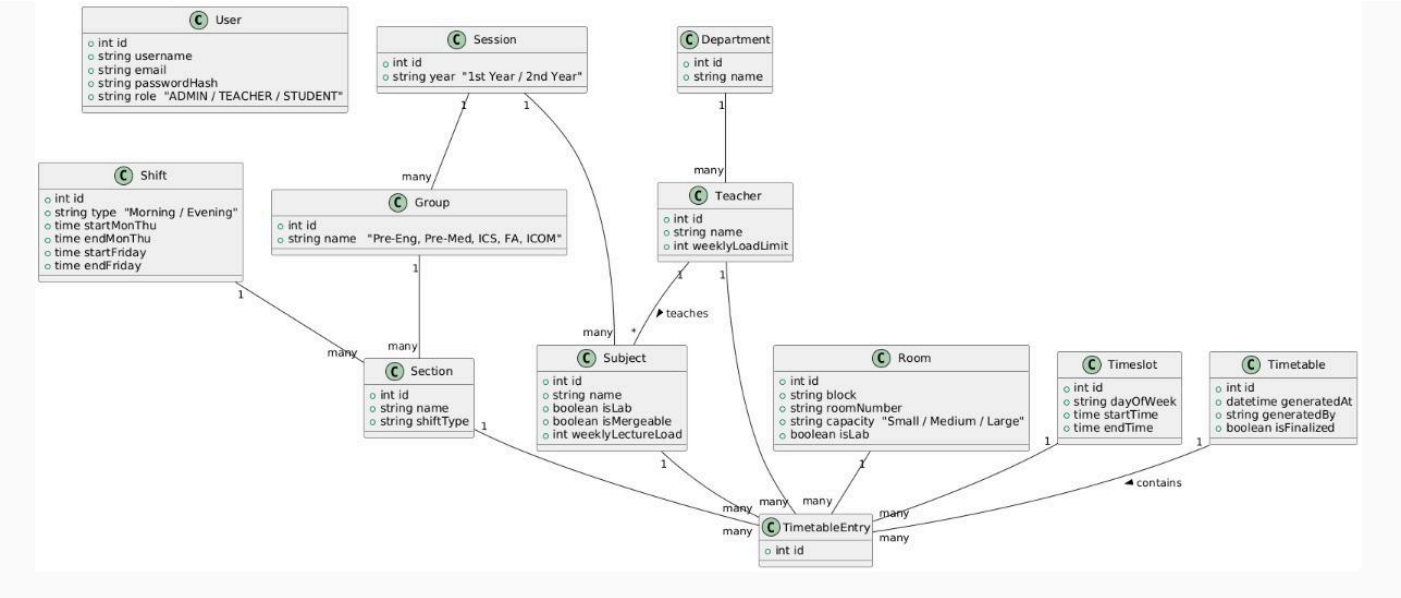
- ***Typical Course of Action***

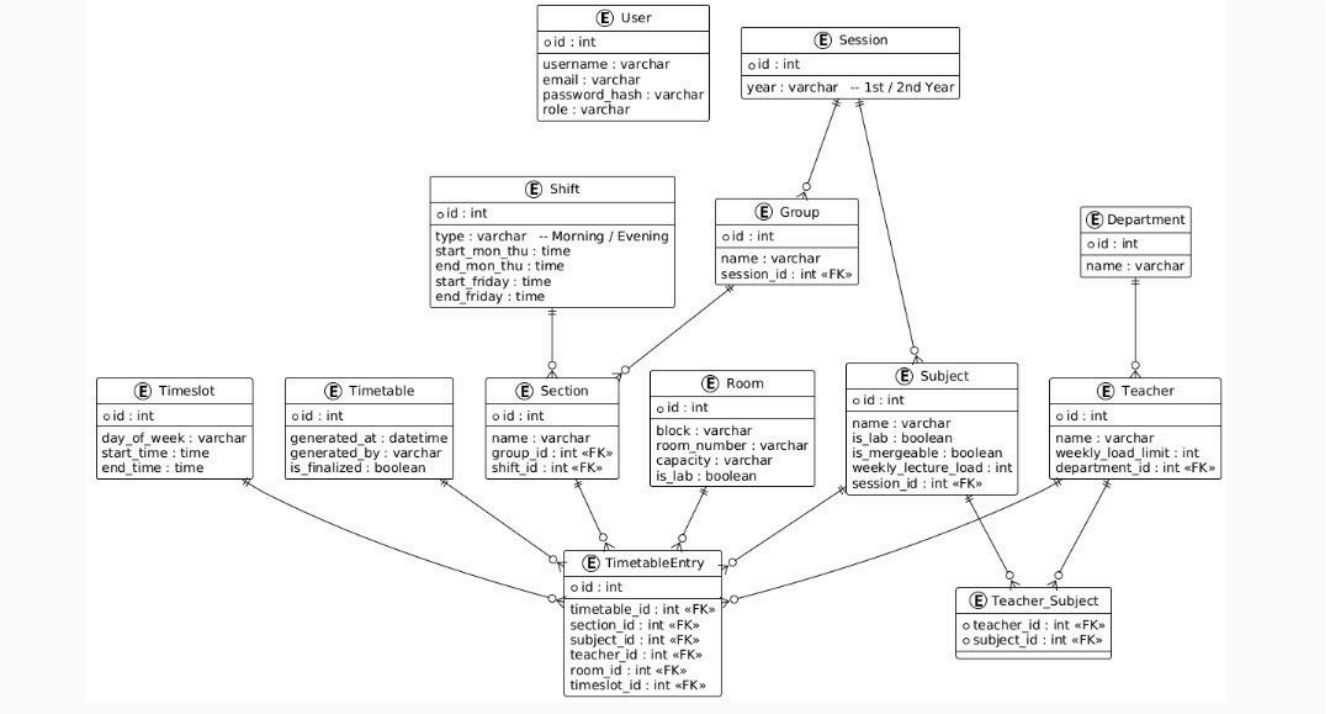
S#	*Actor Action*	*System Response*
1	*User navigates to "Rooms" page.*	*System displays rooms/labs.*
2	*User adds/edits room attributes.*	*System validates and saves.*
3	*–*	*System updates timetable constraints.*

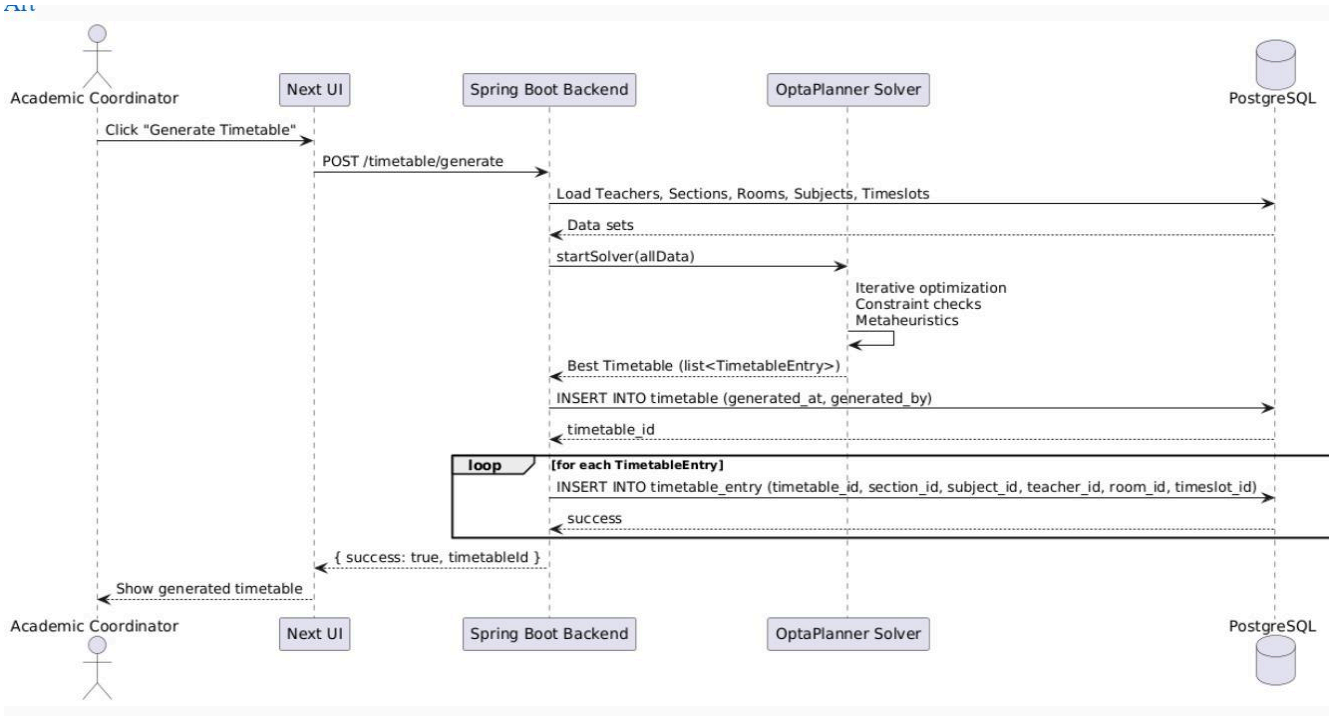
- ***Alternate Course of Action***

S#	*Actor Action*	*System Response*
1	*User sets capacity below 1.*	*System warns: "Capacity must be at least 1."*
2	*User deletes room used in active timetable.*	*System blocks deletion and suggests marking as unavailable.*

○ Requirements Analysis and Modeling







● Nonfunctional Requirements

○ Performance Requirements

Performance requirements are crucial, especially for the core timetable generation feature, which involves complex combinatorial optimization.

- ***Timetable Generation Time:*** *The system must generate a conflict-free or near-optimal timetable for all* ***150 class groups*** *(scheduling over* ***4,500+ lectures per week****) within* ***30 minutes*** *of the Academic Coordinator initiating the process.*
  Rationale: *The current manual process is unsustainable and time-consuming. This time limit ensures the automated system provides a meaningful, timely improvement.*
- ***API Response Time (Schedule Retrieval):*** *API endpoints used for retrieving and displaying individual schedules (Teacher Schedule Viewer, Room Occupancy Grid, Section View) must respond in* ***under 2 seconds****.*
  Rationale: *Ensures a fast, responsive user experience for end-users (Teachers, Students, Administrators).*
- ***Scalability:*** *The system must be able to handle a future increase of* ***20%*** *in the number of class sections (up to ≈ 180 sections) and teachers (≈ 180 teachers) without exceeding the 30-minute generation time or degrading API response times.*
  Rationale: *The system needs to support institutional growth and provide long-term value.*

○ Safety Requirements

Safety in this context relates to preventing data loss, system failure, and potential operational harm resulting from system errors.

- ***Data Integrity and Consistency:*** *The use of* ***PostgreSQL*** *requires that all database transactions must adhere to ACID properties to ensure that a partially completed lecture assignment update (UC-3) or generation process (UC-2) does not corrupt the scheduling data.*
  Safeguards: *Use of Hibernate (JPA) transaction management.*
- ***Conflict Prevention:*** *The core scheduling engine (OptaPlanner) must be designed to never violate a defined* ***Hard Constraint*** *(e.g., preventing a teacher or room from being assigned two lectures at the exact same time). Any failure to prevent a hard conflict is considered a critical safety failure.*
- ***Backup and Recovery:*** *A mechanism must be in place for daily automated backups of the PostgreSQL database. The system must be recoverable to the last known good state within* ***4 hours*** *of a catastrophic failure.*

○ Security Requirements

Security is paramount for protecting sensitive administrative data and ensuring authorized access.

- ***User Authentication:*** *All actors must log in using a secure authentication system. This includes hashing and salting passwords (e.g., using bcrypt) and implementing a secure* ***Forget Password*** *mechanism.*
- ***Authorization (Role-Based Access Control - RBAC):*** *The system must enforce strict role segregation:*
  - ***Academic Coordinators:*** *Full read/write privileges, including entity management, timetable generation, and manual overrides.*
  - ***Teachers/Administration:*** *Read-only access, limited to viewing relevant schedules and analytics.*
- ***Secure Communication:*** *All data transmission between the client (React.js) and the server (Spring Boot REST API) must be encrypted using* ***HTTPS/TLS****.*
- ***Input Validation:*** *All user inputs (UC-1, UC-3) must be validated on both the client-side (JavaScript) and the server-side (Spring Boot) to prevent injection attacks and ensure data cleanliness.*

○ Additional Software Quality Attributes

- ***Adaptability:*** *The system should allow changes in academic rules without needing major code changes.*
- ***Availability:*** *The system should be accessible whenever timetable generation is required.*
- ***Correctness:*** *The system should always generate timetables that follow all compulsory rules.*
- ***Flexibility:*** *Authorized users should be able to manually adjust the timetable when necessary.*
- ***Maintainability:*** *The system should be easy to update and maintain in the future.*
- ***Portability:*** *The system should be able to run on common cloud platforms without special setup.*
- ***Reliability:*** *The system should consistently produce correct timetables for the same inputs and keep the dependencies and compromises consistent for every run of the system.*
- ***Reusability:*** *The scheduling logic should be reusable for other academic programs with minor changes.*
- ***Robustness:*** *The system should handle incorrect or missing data (entering wrong input while declaring sessions, subject, rooms etc) without crashing.*
- ***Usability:*** *The system should be easy to understand and use by non-technical staff and with very clear terminology.*
- ***Scalability:*** *The system should handle an increase in number of classes, teachers, and rooms without major performance issues and should adjust to the strain put on it based on the institutes situation.*

Other Requirements

- ***External Interfaces (Software):*** *The system's primary interface is the* ***REST API*** *provided by the Spring Boot backend for consumption by the React.js frontend.*
- ***Database Requirements:*** *The system requires a production instance of* ***PostgreSQL****. It must be optimized for query speed, particularly for fetching large sets of* `TimetableEntry` *records.*
- ***Internationalization Requirements:*** *At this stage, the system is designed for a single language (****English****) and a single locale (****Pakistan Standard Time - PKT****). Future internationalization is an objective but not a current requirement.*
- ***Reuse Objectives:*** *The logic and architecture of the GA engine, designed using OptaPlanner, are intended to be a reusable module that can be adapted for other educational institutions.*

Revised Project Plan

The project is currently on track, having completed the initial planning and analysis phases. The timeline is updated to reflect progress as of ***November 2025****.*

Current Status

The team has successfully completed ***Requirement Gathering*** *and* ***Requirement Analysis****. The project is now actively transitioning into the* ***System Design & Architecture*** *phase, which is currently* ***in progress****. The next critical phase involves parallel development of rest of the Frontend, Backend, and the core Scheduling Engine.*

Updated Project Milestones and Timeline

The following Gantt chart provides a conceptual representation of the revised project plan, showing completed work and scheduled future tasks.

Milestone	***Original Timeline***	***Status (Nov 2025)***	***Updated Timeline***
1. Requirement Gathering	*September 2025*	***Complete***	*Completed*
2. Requirement Analysis (Finalized SRS)	*September - October 2025*	***Complete***	*Completed*
3. System Design & Architecture (DB Schema, Module Design)	*October - November 2025*	***In Progress (90%)***	*November 2025*

4. Frontend Development (UI, Dashboard, Forms)	*November - December 2025*	***In Progress (80%)***	*December 2025*
5. Backend Development (REST APIs, DB Integration)	*November - December 2025*	***In Progress (30%)***	*December- January 2026*
6. Scheduling Engine Development (OptaPlanner Core)	*November - December 2025*	*Not Started*	*December- January 2026*
7. Integration, Debugging & Testing (Unit, Conflict Module)	*January - February 2026*	*Not Started*	*January - February 2026*
8. Deployment & Production (Cloud Setup, User Manual)	*March 2026*	*Not Started*	*March 2026*

Gantt Chart

Task Name	***Sept 2025***	***Oct 2025***	***Nov 2025***	***Dec 2025***	***Jan 2026***	***Feb 2026***	***Mar 2026***
1. Req. Gathering							
2. Req. Analysis							
3. Design & Arch.							

4. Frontend Dev.							
5. Backend Dev.							
6. Engine Dev.							
7. Integration/Testing							
8. Deployment & Prod.							

References

1. *Red Hat Build of OptaPlanner 8.29 Documentation, "OptaPlanner on Spring Boot: a school timetable quick start guide," Red Hat, Inc. [Online]. Available: https://docs.redhat.com/en/documentation/red_hat_build_of_optaplanner/8.29/html/developing_solvers_with_red_hat_build_of_optaplanner/assembly-business-optimizer-springboot_optaplanner-quickstarts*
2. *OptaPlanner, "School Timetabling Use Case," OptaPlanner.org. [Online]. Available: https://www.optaplanner.org/learn/useCases/schoolTimetabling.html*
3. *S. Thakare, T. Nikam, and M. Patil, "Automated Timetable Generation Using Genetic Algorithm," International Journal of Engineering Research & Technology (IJERT), vol. 9, no. 7, 2020. [Online]. Available: https://www.ijert.org/research/automated-timetable-generation-using-genetic-algorithm-IJERTV9IS070568.pdf*
4. *"Automated Timetable Generator Using Genetic Algorithm," International Journal of Research in Advent Technology (IJRAT), vol. 1, no. 10, Oct. 2013. [Online]. Available: https://ijrat.org/downloads/Vol-1/oct-2013/paper%20id-13201322.pdf*
5. *R. Johnson, Spring Boot in Action, Shelter Island, NY: Manning Publications, 2016. (Supports your Technology Stack Mandate and Backend Development). https://www.manning.com/books/spring-boot-in-action*
6. *IEEE Computer Society, "IEEE Standard for System, Software, and Hardware Verification and Validation," IEEE Std 1012-2016, 2017. (Supports your Appendix B: IV & V Report). https://ieeexplore.ieee.org/document/8055462*
7. *F. Buschmann, R. Meunier, and H. Rohnert, Pattern-Oriented Software Architecture: A System of Patterns, New York: Wiley, 1996. (Supports Section 3.3: Requirements Analysis and Modeling). https://www.wiley.com/en-us/Pattern+Oriented+Software+Architecture%2C+Volume+1%2C+A+System+of+Patterns-p-9780471958697*

8. *A. Banks and E. Porcello, Learning React: Modern Patterns for Developing React Apps, 2nd ed. Sebastopol, CA: O'Reilly Media, 2020. (Supports your Frontend Development and PWA requirements). https://www.oreilly.com/library/view/learning-react-2nd/9781492051718/*
9. *C. Bauers, PostgreSQL 16 High Performance: Expert techniques for query optimization, high availability, and scalability, 3rd ed. Birmingham, UK: Packt Publishing, 2023. (Supports your Database Requirements and Performance sections). https://www.packtpub.com/en-us/product/postgresql-16-administration-cookbook-9781835460580*

Appendix A: Glossary

Term / Acronym	*Category*	*Definition*
ACID Properties	*Technical*	*A set of properties (Atomicity, Consistency, Isolation, Durability) that ensure reliable database transactions.*
Alternate Course of Action	*Documentation*	*A secondary flow in a use-case describing system behavior under exceptional conditions.*
Analytics Dashboard	*UI/UX*	*An interface component presenting aggregated system metrics (e.g., room usage) in graphical form.*
API	*Technical*	*Application Programming Interface; the bridge enabling communication between the React frontend and Spring Boot backend.*
Authorization (RBAC)	*Security*	*Role-Based Access Control; restricting system access based on predefined roles (Admin, Teacher, etc.).*
Chromosome	*Genetic Algorithm*	*An encoded representation of a candidate solution (a potential timetable) used during optimization.*
Cloud Deployment	*Infrastructure*	*Hosting the application on cloud platforms (AWS, Render, Railway) rather than local servers.*

Constraint (Hard)	*Optimization*	*A mandatory rule that **must not** be violated (e.g., a teacher cannot be in two rooms at once).*
Constraint (Soft)	*Optimization*	*A preferential rule that improves quality (e.g., minimizing gaps in a teacher's day) but may be violated.*
Constraint Solver	*Technical*	*The software component (OptaPlanner) that generates solutions while satisfying defined rules.*
Crossover	*Genetic Algorithm*	*A GA operation where two candidate solutions are combined to produce a new "offspring" solution.*
Double-Period Lab	*Institutional*	*A laboratory session that must be scheduled across two consecutive time slots.*
Fitness Function	*Genetic Algorithm*	*A mathematical function used to evaluate and "score" the quality of a generated timetable.*
Genetic Algorithm (GA)	*Optimization*	*An evolutionary technique inspired by natural selection used to solve complex scheduling problems.*
Hibernate (JPA)	*Technical*	*An Object-Relational Mapping framework that links Java objects to the PostgreSQL database.*
IV & V	*Documentation*	*Independent Verification & Validation; processes to ensure the system is built correctly and meets requirements.*

Metaheuristic	*Optimization*	*A high-level algorithmic framework (e.g., Tabu Search) used to find near-optimal solutions efficiently.*
Mutation	*Genetic Algorithm*	*A GA operation introducing random changes to a solution to maintain diversity and find better paths.*
OptaPlanner	*Technical*	*The open-source AI constraint satisfaction engine used as the core scheduling logic for Slotify.*
PWA	*UI/UX*	*Progressive Web App; a web application that provides app-like functionality and mobile compatibility.*
REST API	*Technical*	*A stateless web service architecture used for communication between client and server.*
Shift Timing	*Institutional*	*Predefined academic session periods, specifically Morning and Afternoon shifts at FCC.*
Solver Runtime	*Performance*	*The total time taken by the optimization engine to generate a complete timetable solution.*
Spring Boot	*Technical*	*The Java-based framework used for developing the standalone backend application.*
Timeslot	*Institutional*	*A specific unit of time defined by a day and a lecture period (e.g., Monday, Period 1).*

Workload Balance	*Optimization*	*The equitable distribution of teaching hours among faculty members to ensure fairness.*

Appendix B: IV & V Report

(Independent verification & validation)

IV & V Resource

Name Signature

S#	Defect Description	Origin Stage	Status	Fix Time	
				Hours	Minutes
..					

Table 3: List of non-trivial defects